

Capstone Exercise Part I:

Optimizing 2D Image Convolution using SIMD

I. Overview

In this lab, you will implement and optimize a program that performs 2D image convolution on a 24-bit RGB bitmap image using a fixed 3×3 kernel. Your goal is not only to produce a functionally correct convolution program, but also to study how performance changes when optimization techniques such as SIMD and other techniques discussed in class are applied.

You will create **three separate implementations** of the same program, each representing a different optimization level. Afterward, you will compare their execution times and discuss the speedups you achieved in a written report.

This exercise is intended to give you hands-on experience in performance-oriented programming, especially in the use of low-level CPU optimizations.

II. Brief Introduction to 2D RGB Image Convolution

A digital image can be viewed as a 2D grid of pixels. In a 24-bit RGB bitmap image, each pixel contains three color components: Red, Green, and Blue, each typically stored in 8 bits ([0, 255]).

A 2D convolution applies a small matrix called a **kernel** or **filter** over the image. For a 3×3 kernel, the program considers a 3×3 neighborhood around each pixel. That is, if the pixel in the image that we are considering is located at coordinates (x, y) , then the pixels that will be considered are located at the following coordinates:

$$\begin{bmatrix} (x - 1, y - 1) & (x, y - 1) & (x + 1, y - 1) \\ (x - 1, y) & (x, y) & (x + 1, y) \\ (x - 1, y + 1) & (x, y + 1) & (x + 1, y + 1) \end{bmatrix}$$

For each output pixel, the corresponding neighboring input pixels are multiplied by the kernel values, and the results are summed. It is essentially a dot product operation done on a matrix. For RGB images, this operation is applied separately to each color channel. In other words, the red value of the output pixel is computed from the red values of neighboring pixels, and similarly for green and blue.

In this lab, the convolution must use a **stride of 1**, which means the filter moves one pixel at a time horizontally and vertically across the entire image. As the filter moves, the dot product operation is performed to produce an output pixel. Because of this, the output image would have the same dimensions as the input image. At the borders of the image, some neighboring pixels needed by the kernel will fall outside the valid image area (e.g., at pixel $(0, 0)$, there is no valid pixel data at the location $(-1, -1)$). You may choose any consistent border handling method. The easiest choice is **zero-padding**, where out-of-bounds pixels are treated as zero. Whatever method you choose must be used consistently across all four implementations.

(You may watch this video for a more visualized explanation of 2D convolution: [📺 But what is a convolution?](#))

III. Input and Output handling

Your program must accept as input a **24-bit RGB bitmap file**. The image dimensions may vary, so your program must be able to:

- determine the image width and height from the file,
- allocate memory dynamically based on the image size,
- perform the convolution, and
- write the resulting image back to a bitmap file.

The convolution kernel is **fixed at 3×3**, but the **actual numerical values of the kernel are up to you**. You may use a blur, sharpen, edge-detection, emboss, or any other valid 3×3 filter. Choose a kernel that is meaningful and document it clearly in your report. You may opt to have several preset kernels for the user to choose from; just provide the corresponding instructions for the user in the documentation.

Since the output image must have the same dimensions as the input image, your implementation must preserve width and height. You should also ensure that the resulting pixel values remain valid for 8-bit color channels. If necessary, clamp values to the range 0 to 255 before writing them back to the output image.

Your implementations should prioritize correctness first, then optimization.

IV. Implementation and Optimization Details

You are required to produce **three separate source codes**, each implementing the same convolution task but using different optimization strategies.

1. Baseline Program

The first version is the baseline or “vanilla” implementation. This version must use ordinary **for loops only** and must not use SIMD instructions, multithreading, or any other explicit optimization technique.

This program serves as the reference point for correctness and performance. Make sure this version works correctly before moving on to the optimized versions. Any later performance comparison will be based on this implementation.

2. Memory Access Optimizations

The second version must apply memory access optimizations as discussed in the lectures, most of which you have tested and benchmarked in previous software exercises. You have the freedom to choose which optimization techniques are most suitable for your program.

This is also where you can improve your program using high-performance coding “hacks” or best practices (in case you did not write your baseline program this way).

2. SIMD-Optimized Program

The third version must optimize the baseline implementation using **SIMD**. This means you should use **x86 intrinsics** to exploit data-level parallelism. No multithreading should be used in this version.

The goal here is to process multiple data elements in parallel within a single CPU instruction. Since this is an RGB image convolution, careful handling of pixel layout and channel data will be necessary. You may reorganize computations as needed, as long as the output remains functionally correct and equivalent to the baseline under the same kernel and border handling method.

A sample SIMD program written in C is available on UVLe.

You will most likely use **C** for this exercise, especially since the sample code for SIMD, and the examples discussed in the lecture, are written in C. You may use another programming language only if you are able to apply equivalent optimization techniques in that language and satisfy all requirements of the exercise. Regardless of language, all three implementations must remain comparable and must solve the same problem under the same conditions.

All three programs must produce equivalent results, subject to the same kernel and border handling policy. Before claiming any speedup, make sure the output of your optimized programs is correct. A faster program that produces incorrect output will not be considered a valid optimization. You are encouraged to verify correctness by testing your output images against the baseline implementation.

V. Evaluation Metrics

In your report, you must make a comparative analysis of the three versions of the code in terms of code complexity and code balance. Discuss how the optimizations you applied in versions two and three of your program affect code complexity and code balance.

For evaluating the speedup gained through your optimizations, you will measure the **walltime** of each implementation and compute the **relative speedup** compared to the baseline. For example, if the baseline takes T_b seconds, and an optimized version takes T_o seconds, then the speedup is $\frac{T_b}{T_o}$.

You should perform timing carefully and consistently. Make sure that the timing method you use measures the convolution portion of the program in a fair and repeatable manner. If file I/O is included in your timing, state that clearly in your report. If it is excluded, state that as well. To make your measurements more reliable, you may run each program multiple times and use an average or best-case time, provided you describe your method clearly.

You are expected to test your implementations on your EEEl virtual machines so that performance results are obtained under a controlled and consistent setup.

V. Written report

You are required to submit a written report discussing your implementations and results. You should provide a brief description of how to run your programs, so that the checker can also verify your implementations. Your report should include a description of each of the three implementations.

For the optimized versions, explain how the optimizations were applied. For example, describe what part of the computation was reordered to improve memory access patterns, or which part of the computation was vectorized in the SIMD version. Include execution times and computed speedups for all versions. Present the results clearly, preferably in a table and/or graph. Finally, discuss your observations. Which optimization gave the largest improvement? Did applying SIMD provide the expected benefit? Were there any limitations, bottlenecks, or implementation challenges?

VI. Deliverables

Submit the following:

1. Source code for the baseline implementation
2. Source code for the memory access optimized implementation
3. Source code for the SIMD-only implementation
4. Your written report containing implementation details and performance results

Make sure your submissions are clearly labeled and organized.